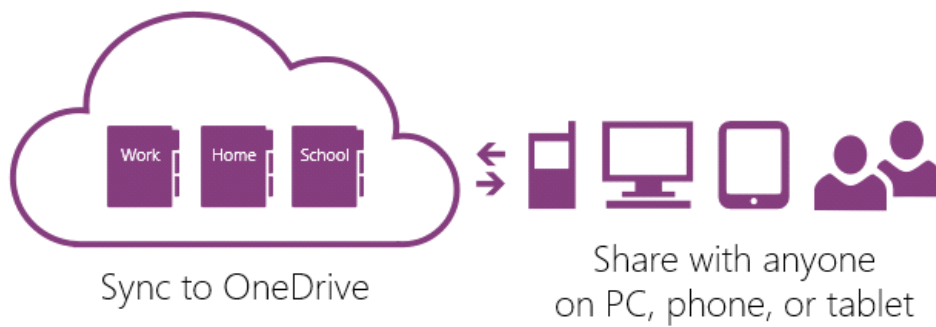


OneNote: one place for all of your notes



 [Watch the 2 minute video](#)

1. Take notes anywhere on the page

Write your name here



2. Get organized

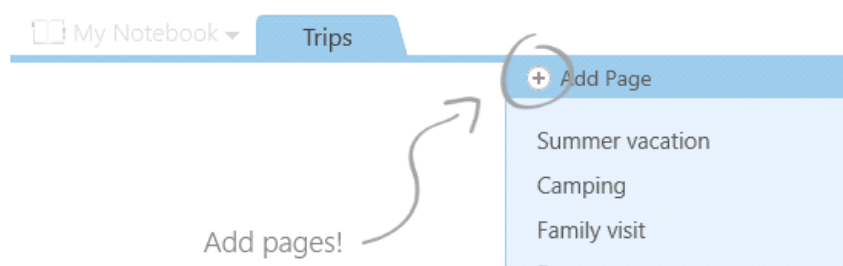
You start with "My Notebook" - everything lives in here

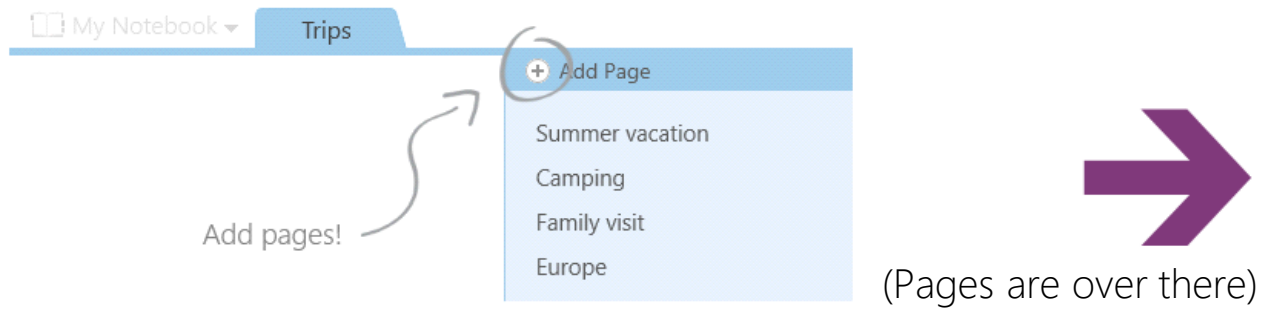


Add **sections** for activities like:

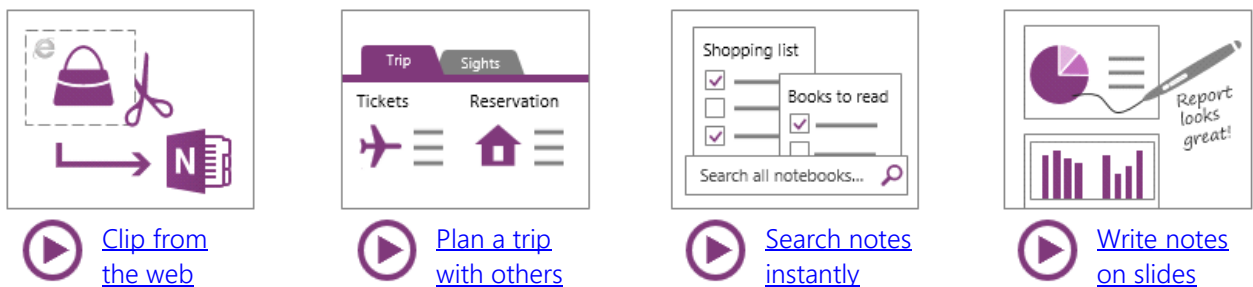


Add **pages** inside of each section:



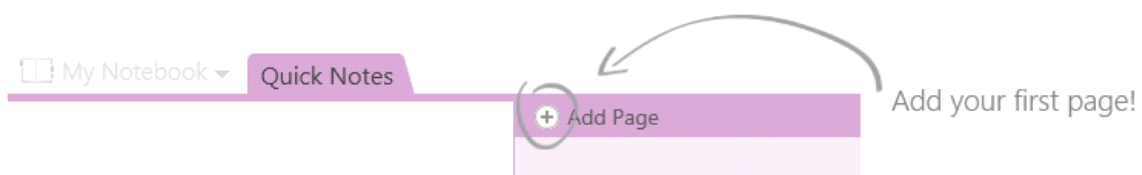


3. For more tips, check out 30 second videos

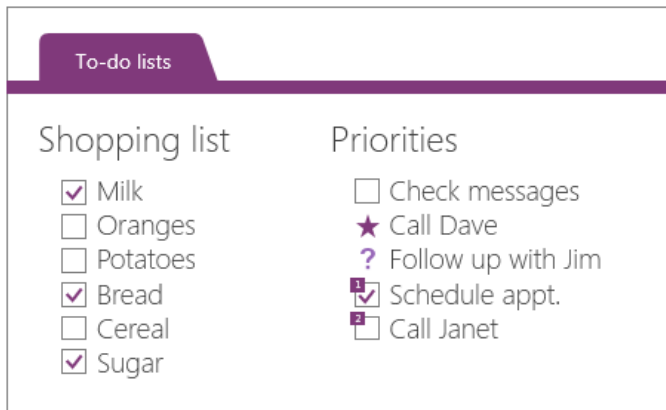


4. Create your first page

You're in the Quick Notes section - use it for random notes

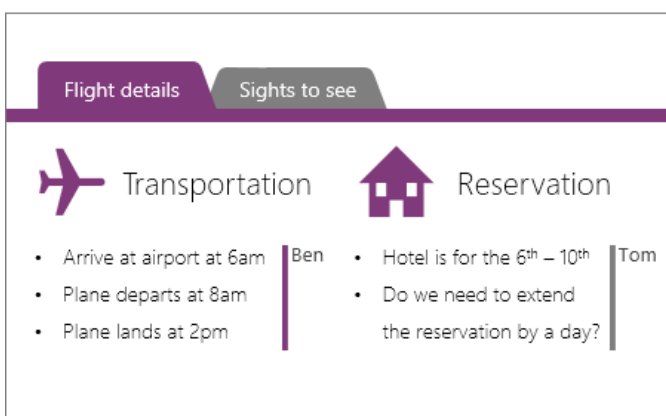


OneNote Basics



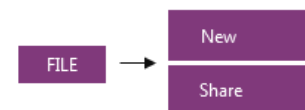
Remember everything

- Add Tags to any notes
- Make checklists and to-do lists
- Create your own custom tags



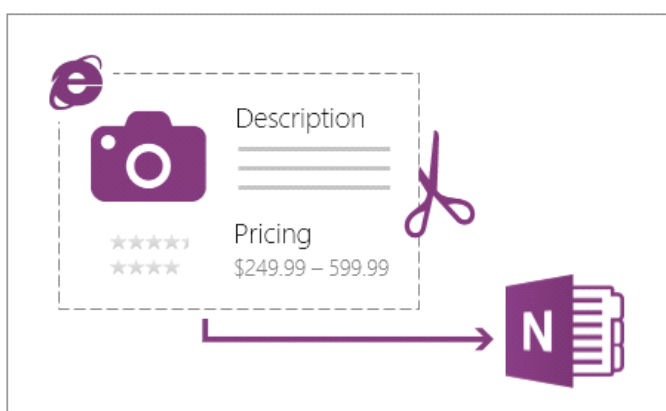
Collaborate with others

- Keep your notebooks on OneDrive
- Share with friends and family
- Anyone can edit in a browser



Keep everything in sync

- People can edit pages at the same time
- Real-Time Sync on the same page
- Everything stored in the cloud
- Accessible from any device



Clip from the web

- Quickly clip anything on your screen
- Take screenshots of products online
- Save important news articles

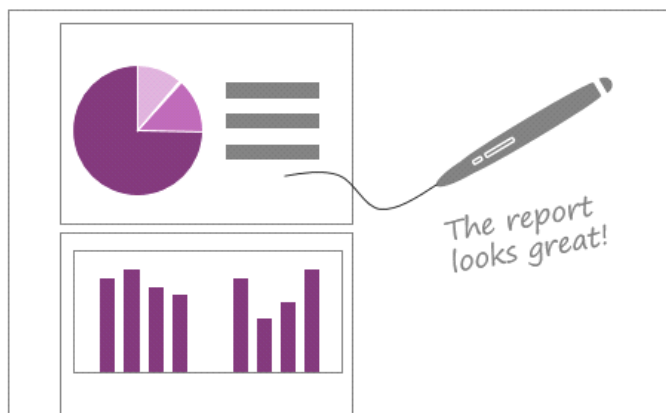


Sunday retreat			
	Attending?	Overnight?	Vegetarian?
Chris	Yes	Yes	No
Molly	No	No	No
Peter	Yes	No	Yes
Samuel	Yes	Yes	Yes
Stacy	Yes	No	No

A
Z ↓

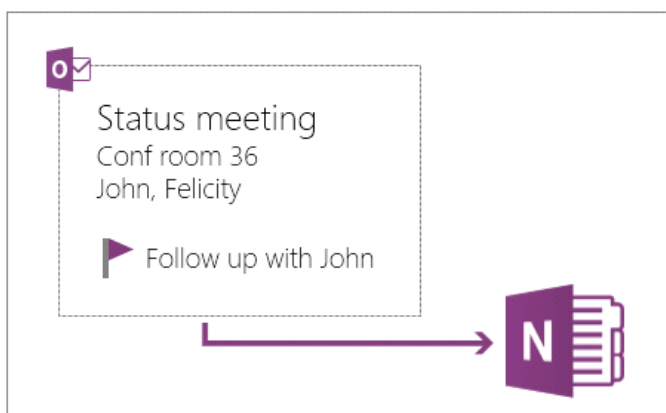
Organize with tables

- Type, then press TAB to create a table
- Quickly sort and shade tables
- Convert tables to Excel spreadsheets



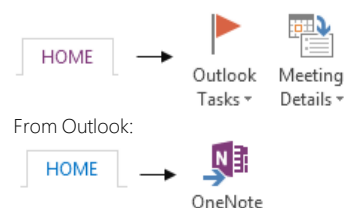
Write notes on slides

- Send PowerPoint or Word docs to OneNote
- Annotate with a stylus on your tablet
- Highlight and finger-paint



Integrate with Outlook

- Take notes on Outlook or Lync meetings
- Insert meeting details
- Add Outlook tasks from OneNote

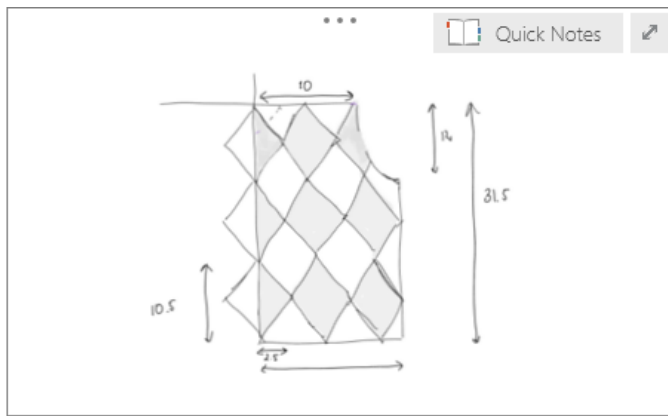


Quarter 1 revenue			
	Sales	Revenue	Expenses
Scott	4	5	3
James	2	1	4

Add Excel spreadsheets

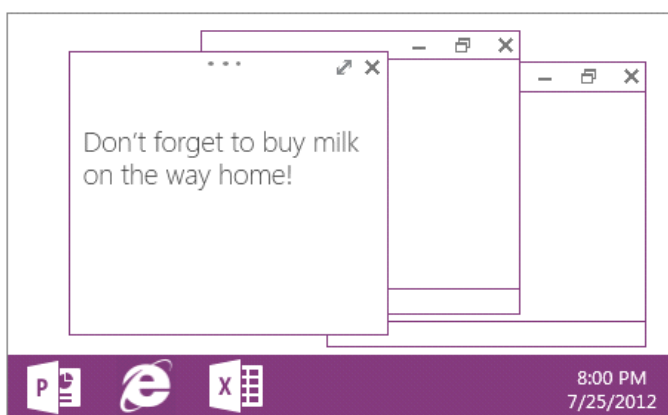
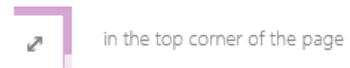
- Track finances, budgets, & more
- Preview updates on the page





Brainstorm without clutter

- Hide everything but the essentials
- Extra space to focus on your notes



Take quick notes

- Quickly jot down thoughts and ideas
- They go into your Quick Notes section



Computer Fundamentals Day 1

30 January 2026 12:22 PM

❖ **Contiguous** : Continuous memory block touching each other.

❖ **Ram segments** : (Logical divisions of a program's address space)

Code segment	Data segment	Stack segment	Heap segment	Command line arguments	Environment variables or system variables
--------------	--------------	---------------	--------------	------------------------	---

The 4 Primary Memory Segments

1. Code Segment (Text Segment)

This is where the actual **compiled machine code** (the instructions) resides.

- **Characteristics:** Usually read-only to prevent a program from accidentally modifying its own instructions.
- **Life Span:** Exists for the entire duration of the program.

2. Data Segment

This is divided into two sub-sections based on whether variables have a starting value:

- **Initialized Data Segment:** Stores global, static, and constant variables that have a predefined value (e.g., `int x = 10;`).
- **Uninitialized Data Segment :** It stores global and static variables that are initialized to zero or have no initialization in the source code (e.g., `int y;`).

3. Stack Segment

The stack is used for **static memory allocation**. It follows a Last-In, First-Out structure.

[Refer this Best Example ☺](#)

- **What it stores:** Local variables, function parameters, and return addresses.
- **How it works:** When a function is called, a "stack frame" is added. When the function finishes, that frame is "popped" and the memory is immediately available.
- **Note:** It has a fixed size; if you use too much (like with infinite recursion), you get the dreaded **Stack Overflow**.

4. Heap Segment

The heap is used for **dynamic memory allocation** during runtime.

- **What it stores:** Objects or data structures created manually by the programmer (using `malloc()` in C or `new` in C++).
- **How it works:** It is much larger than the stack but requires manual management. If you allocate memory here and forget to free it, you create a **memory leak**.

❖ **Storage Class Specifier :**

- 1) Static
- 2) Auto
- 3) Extern
- 4) Register
- 5) Volatile , typedef (not exactly a storage class specifier)

❖ **Memory Leak :**

The program "claims" a piece of RAM, loses the "address" (pointer) to that memory, and never tells the computer it's done using it. Over time, these un-reclaimed chunks accumulate, reducing the total available RAM until the system slows down or the program crashes entirely.

❖ Hyperlink for slideshare :

[REFER SLIDESHARE BY NELSON SIR](#)

- 1) Write a program using Malloc() for dynamic array (done in VS code)
- 2) Latest data Structures which people are using

Ans:-

Data	Primary Use	Why it's "Latest/Trending"
Vector Embeddings	AI & Semantic Search	Represents meaning, not just keywords.
Tensors	Machine	Optimized for GPU/TPU parallel processing.
Knowledge Graphs	AI Context & Logic	Links data points for better reasoning.
Bloom Filters	Caching & Big Data	Huge memory savings for membership checks.
LSM Trees	High-speed logging (data	Best for modern, write-intensive cloud apps.

Command line arguments

30 January 2026 12:48 PM

❖ Command line arguments :

To explain command-line arguments, we can look at a simple C program that breaks down how the computer "sees" the extra words you type after a command.

❖ The Code (C Language):

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    // 1. Show the total count
    printf("Total arguments: %d\n", argc);

    // 2. Loop through and print each argument
    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n", i, argv[i]);
    }

    return 0;
}
```

How it Works:

- ❖ **argc (Argument Count):** An integer that tells the program how many items were typed on the command line, including the program's own name.
- ❖ **argv (Argument Vector):** An array of strings (text) where each word is stored.

argv[0] is always the name of the program.
argv[1] is the first real argument you provided.

Example Execution:

If you compile this as myprog and run it like this:

```
./myprog apple banana 42
```

The program will output:

```
Total arguments: 4
Argument 0: ./myprog
Argument 1: apple
Argument 2: banana
Argument 3: 42
```

(Note: Even numbers are received as strings and must be converted using functions like `atoi()` if you want to do math with them).

Frame

30 January 2026 01:20 PM

- **Frame** :

Fundamental unit of data transmission at data linker layer

Now what is data linker layer??

It ensures reliable node to node data transfer across a physical network by encapsulating packets from the network layer into frames.

Spaghetti problem

30 January 2026 01:22 PM

- **The Main Culprit: The goto Statement**

The most common cause of spaghetti code in C is the goto statement. While goto has legitimate uses (like breaking out of deeply nested loops), it allows a programmer to jump to any labeled part of a function, bypassing the natural top-to-bottom logic.

In normal language according to me XD code which is readable to you but not accurately readable to others
Due to various jumps in the code.

Characteristics of Spaghetti Code

- **Lack of Structured Control:** Ignoring if-else, while, and for loops in favor of manual jumps.
- **Excessive Global Variables:** Using global variables to pass data between distant, unrelated parts of the code.
- **"Golden Hammer" Functions:** Massive functions (1000+ lines) that try to do everything at once.
- **High Coupling:** Changing a single line in one part of the file breaks something completely unrelated in another.

❖ **Solution:** STRUCTURED PROGRAMMING

Why do we have three loops ?

30 January 2026 01:26 PM

In programming, we have three different loops because **different problems require different "checking" points**. While you could technically use a while loop for everything, having specialized tools makes your code cleaner, more readable, and less prone to errors.

Think of it like choosing a vehicle: a bike, a car, and a truck all get you from A to B, but you wouldn't use a truck to pick up a loaf of bread.

If we only had while, every for loop would require three separate lines of code (creating the counter, the loop, and incrementing), which leads to the **Spaghetti Problem** we discussed earlier. Specialized loops keep the "scaffolding" of your code organized.

1. The for Loop: The "Accountant"

Use a for loop when you know **exactly how many times** you need to repeat a task before you even start.

- **Logic:** Everything is packed into one line: Initialization, Condition, and Increment.
- **Best for:** Iterating through arrays, counting numbers, or fixed repetitions.